

Estrutura de Dados I

Recursividade

- Uma função recursiva é uma função que resolve uma parte pequena de um problema, e que, para resolver o resto do problema, chama ela mesma.
- O conceito é matemático mas se aplica muito bem à programação. Um exemplo vindo da matemática é a definição dos números Naturais (inteiros positivos):
 - 0 é um inteiro natural
 - Se x é um inteiro natural, $x+1$ é um inteiro natural

- Um exemplo fácil de programar é o fatorial. A definição matemática é:
 - $\text{Fatorial}(x) = \text{produto dos inteiros de } 1 \text{ a } x$
 - Por exemplo, $\text{Fatorial}(5) = 1 * 2 * 3 * 4 * 5$.
- Em outras palavras:
 - Se $x = 1$, $\text{Fatorial}(x) = 1$
 - Senão, $\text{Fatorial}(x) = x * \text{Fatorial}(x-1)$

```
#include <stdio.h>
```

```
int fatorial(int n) {  
    if (n == 1) {                // caso base  
        return 1;  
    }
```

```
    return n * fatorial(n - 1);  // chamada recursiva  
}
```

```
int main() {  
    printf("%d", fatorial(5));  
    return 0;  
}
```

- Quando uma função chama ela mesma, um novo espaço é reservado na pilha para a execução da nova função, e o ponteiro do programa, que indica qual linha do programa está sendo executado, pula para o início da função.
- O código executado é o mesmo, mas com um conjunto de variáveis e parâmetros novos, num espaço novo na pilha.
- Na hora do return, a função é desempilhada e volta para onde estava, descartando suas variáveis e parâmetros.
- O programa continua onde tinha parado com suas variáveis e parâmetros antigos.

Onde recursão é usada?

- Torre de Hanoi
- Busca em árvores
- Algoritmos de grafos
- Percorrer diretórios
- QuickSort
- MergeSort
- Fibonacci

Toda recursão precisa de:

- um caso base;
- uma chamada para ela mesma.

- Partes importantes de uma função recursiva:

```
int Fatorial(int x)
```

```
{  
  if (x == 1)  
  {  
    return 1;  
  }
```

Condição de parada das chamadas recursivas.

```
else
```

```
{  
  return x * Fatorial(x-1);  
}
```

Chamada a ela mesma

```
}
```

- Cuidado com Recursões Infinitas:

```
int Fatorial(int x)
{
    if (x == 1)
    {
        return 1;
    }
    else
    {
        return x * Fatorial(x);
    }
}
```

```
int LoopInfinito(int x)
{
    return LoopInfinito(x);
}
```

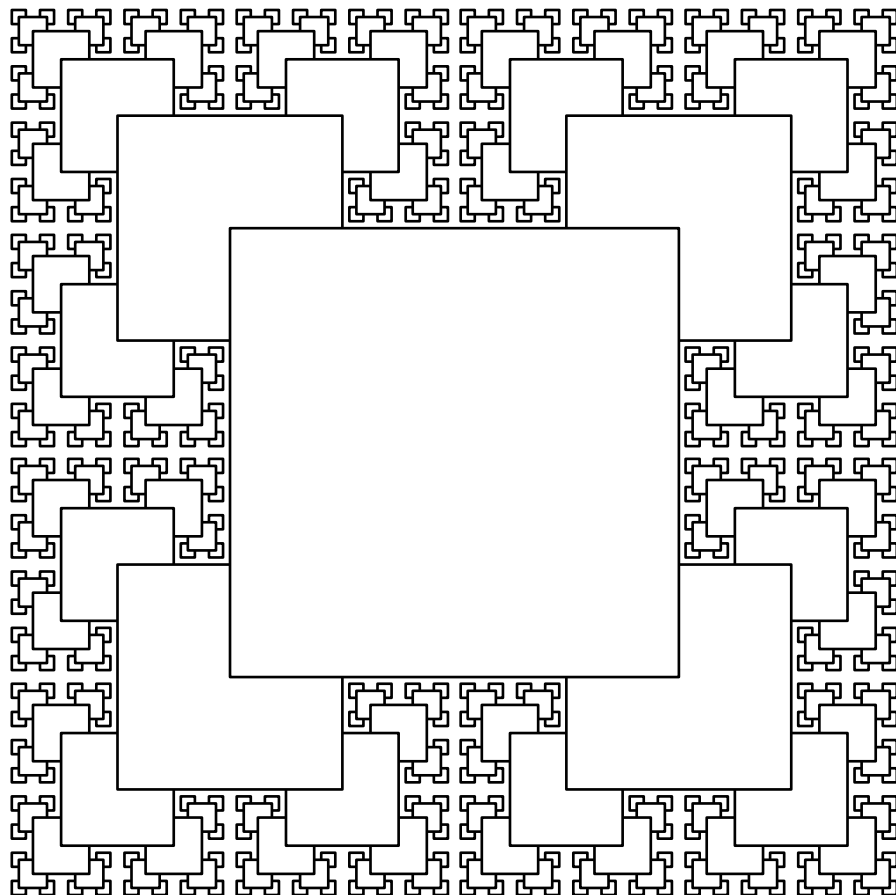

- Solução Recursiva & Não Recursiva:

```
int Fatorial(int x)
{
    if (x == 1)
    {
        return 1;
    }
    else
    {
        return x * Fatorial(x-1);
    }
}
```

```
int Fat (int n)
{
    int f;
    f = 1;
    while(n > 0)
    {
        f = f * n;
        n = n - 1;
    }
    return f;
}
```

- A recursividade nem sempre é a melhor solução, mesmo quando a definição matemática do problema é feita em termos recursivos;
- Recursividade vale a pena para algoritmos complexos, cuja a implementação iterativa é complexa e normalmente requer o uso explícito de uma pilha.
Exemplos:
 - Dividir para Conquistar (Ex. Quicksort)
 - Caminhamento em Árvores (pesquisa, backtracking)

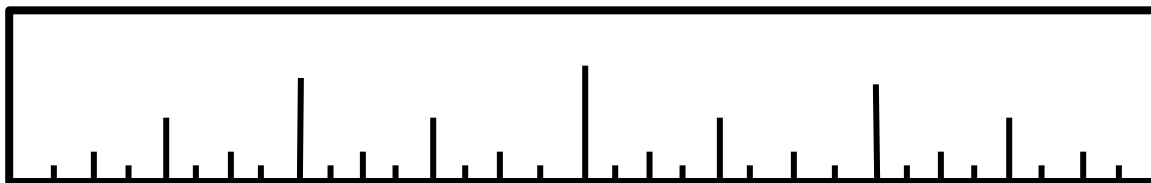
- Outro Exemplo:



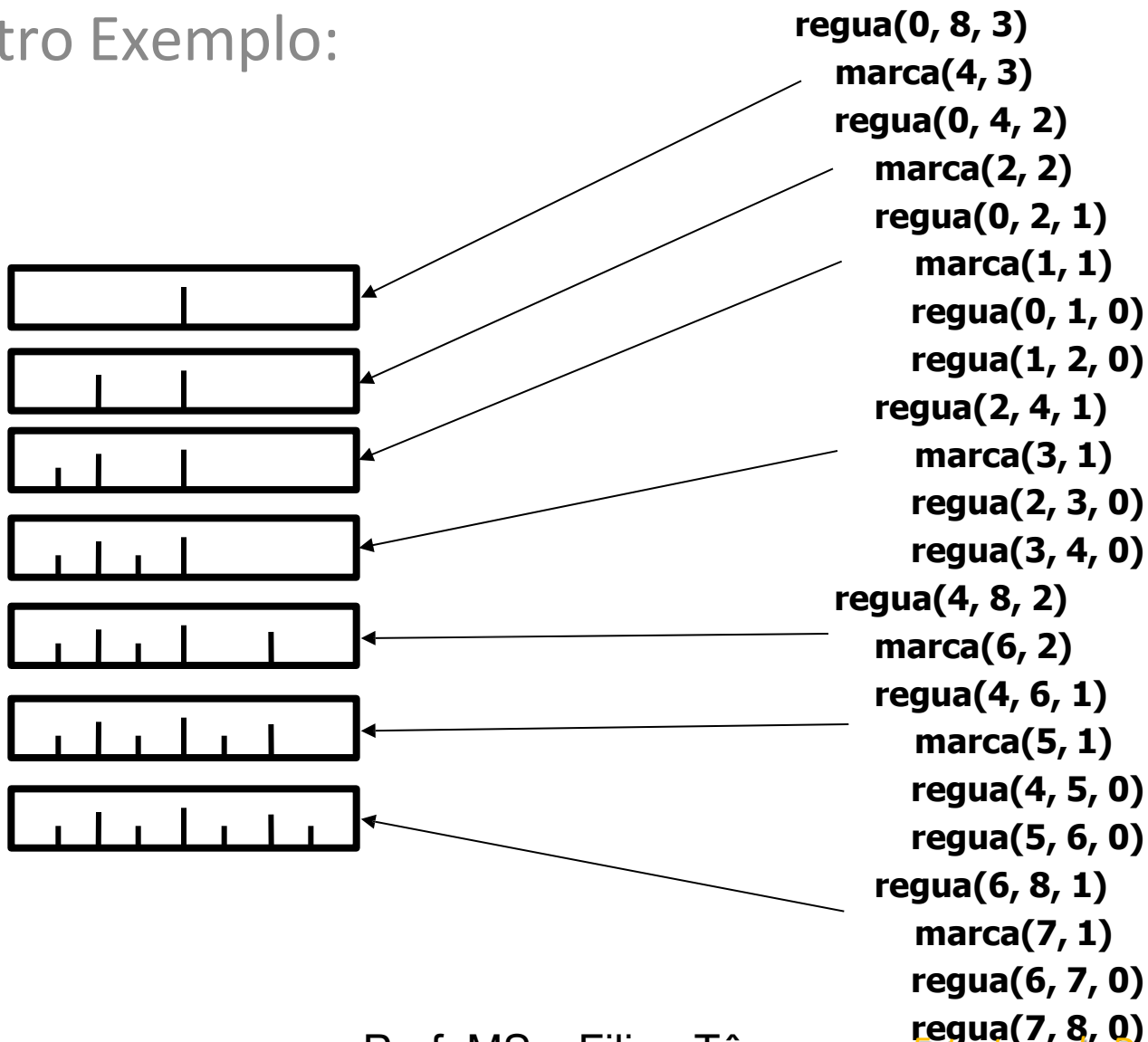
```
void estrela(int x,int y, int r)
{
    if ( r > 0 )
    {
        estrela(x-r, y+r, r div 2);
        estrela(x+r, y+r, r div 2);
        estrela(x-r, y-r, r div 2);
        estrela(x+r, y-r, r div 2);
        box(x, y, r);
    }
}
```

- Outro Exemplo:

```
int regua(int l,int r,int h)
{
    int m;
    if ( h > 0 )
    {
        m = (l + r) / 2;
        marca(m, h);
        regua(l, m, h - 1);
        regua(m, r, h - 1);
    }
}
```



- Outro Exemplo:



- Outro Exemplo: Crie uma função recursiva que calcula a potência de um número

```
int pot(int base, int exp)
{
    if (!exp)
        return 1;

    /* else */
    return (base*pot(base, exp-1));
}
```

O(n)